



**UNIVERSITY OF
PORTSMOUTH**

Coursework for Intelligent Data and Text Analytics (2024/25)

Student Id: 2302988

Table of content

1. INTRODUCTION	3
2. Preprocessing	3
a. Removing punctuation.....	3
b. Removing Numbers:.....	4

c.	Changing text to lower case:	4
d.	Remove stop words.....	4
e.	Lemmatization with POS:.....	5
3.	Classification	5
	Splitting the data:	6
	Transform text into BoW with count features:	6
a.	Random Forest:	6
b.	Naive Bayes:	7
c.	SVM:	8
d.	XGboost:	9
	Comparision:	10
	Best Model: Random Forest	11
	Worst Model: Naive Bayes	12
4.	Classification using a BERT with fine – tuning.....	12
a.	Choose a BERT model to fine-tune and its corresponding pre-processing model	12
	Apply the pre-processing model	13
	Build the classification model	13
	Visualise the model structure.....	14
	Define parameters and optimizer for fine-tuning.....	14
	Fine-tune the BERT model.....	14
	Visualize structure after fine-tuning	15
	Train the model	15
	Plot the accuracy and loss over time.....	16
	Performance metrics.....	16
b.	Bert Model 2:	17
	Importing Libraries:	17
	Setting Up the Model Architecture	17
	Compiling the Model	17
	Training the Model.....	18
	Making Predictions	18
	Evaluating the Model	18
	Comparision of Bert model 1 and model 2:	19
5.	Topic Detection:	20

a. LDA.....	20
Identified Topics and Common Themes	21
Commonalities across the topics.....	22
Findings.....	22
b. LSA	22
Comparision of LDA and LSA:	23
Reference :	25

1.INTRODUCTION

This report details natural language processing (NLP) and machine learning techniques used to analyse the Amazon product review dataset to yield useful insights. This task contains preprocessing steps for text, topic detection using LDA and LSA, text classification based on sentiment prediction, and visualization of insights. The merits of each approach will be discussed and compared with the contributions of these analysis goals.

2.Preprocessing

a. Removing punctuation.

Before	After	Description
So there is no way for me to plug it in here in the US unless I go by a converter.	So, there is no way for me to plug it in here in the US unless I go by a converter	Punctuation marks like commas and periods are removed to simplify the text while preserving the sentence structure and meaning. This helps standardize the text for further processing.
Good case, Excellent value.	Good case Excellent value	Commas are removed to normalize the phrase. The text becomes more concise while maintaining readability, making it suitable for tokenization and feature extraction.
Tied to charger for conversations lasting more than 45 minutes.MAJOR PROBLEMS!!	Tied to charger for conversations lasting more than 45 minutesMAJOR PROBLEMS	Exclamation marks and colons are removed to eliminate unnecessary punctuation. The processed text becomes clean and uniform, which is beneficial for machine learning models.

b. Removing Numbers:

Before	After	Description
Tied to charger for conversations lasting more than 45 minutesMAJOR PROBLEMS	Tied to charger for conversations lasting more than minutesMAJOR PROBLEMS	Numbers like '45' are removed from the text while retaining the surrounding context. This ensures that numerical values do not interfere with text processing when they are irrelevant to the analysis.
If the two were seperated by a mere 5 ft I started to notice excessive static and garbled sound from the headset	If the two were seperated by a mere ft I started to notice excessive static and garbled sound from the headset	The number '5' is removed while maintaining the sentence's meaning. This step is useful for standardizing text when numerical information is not essential for classification tasks.
Ive owned this phone for 7 months now and can say that its the best mobile phone Ive had	Ive owned this phone for months now and can say that its the best mobile phone Ive had	The number '7' is removed from the sentence. This simplifies the data by eliminating numeric values, ensuring consistent preprocessing for downstream tasks.

c. Changing text to lower case:

Before	After	Description
Tied to charger for conversations lasting more than minutesMAJOR PROBLEMS	tied to charger for conversations lasting more than minutesmajor problems	All uppercase letters are converted to lowercase (e.g., 'MAJOR PROBLEMS' to 'major problems') to standardize the text. This ensures uniformity and avoids treating words with different cases as separate entities during analysis.
So there is no way for me to plug it in here in the US unless I go by a converter	so there is no way for me to plug it in here in the us unless i go by a converter	The sentence is transformed entirely to lowercase. Words like 'So' and 'US' are converted to 'so' and 'us' respectively, making the text consistent for NLP tasks where case sensitivity is not needed.
I advise EVERYONE DO NOT BE FOOLED	i advise everyone do not be fooled	Uppercase words like 'EVERYONE' and 'DO NOT BE FOOLED' are changed to 'everyone' and 'do not be fooled,' creating a uniform text format and improving compatibility with machine learning models.

d. Remove stop words

Before	After	Description
so there is no way for me to plug it in here in the us unless i go by a converter	way plug us unless go converter	Stop words like 'so,' 'there,' 'is,' 'no,' 'for,' 'to,' 'in,' 'the,' and 'a' are removed. This reduces the sentence to its essential terms: 'way plug us unless go converter,' making the text more concise and focused on key information.
if you have several dozen or several hundred contacts then imagine the fun of sending each of them one by one	several dozen several hundred contacts imagine fun sending one one	Common stop words such as 'if,' 'you,' 'have,' 'and,' 'of,' 'each,' and 'by' are eliminated. The resulting sentence retains meaningful words like "several dozen," 'contacts,' and 'imagine,' making it more relevant for analysis.
he was very impressed when going from the original battery to the extended battery	impressed going original battery extended battery	Stop words like 'he,' 'was,' 'very,' 'when,' 'going,' 'from,' 'to' and 'the' are removed, leaving the essential words: 'impressed going original battery extended battery,' which highlight the key ideas.

e. Lemmatization with POS:

Before	After	Description
went motorolas website followed directions could get pair	go motorolas website follow direction could get pair	The sentence is simplified by converting 'went' to its base form 'go' and removing redundancy from 'followed directions' to 'follow direction.' The focus is on maintaining the semantic meaning while reducing grammatical complexity.
ive owned phone months say best mobile phone ive	ive own phone month say best mobile phone ive	Verb forms are normalized by changing 'owned' to 'own,' and plural nouns like 'months' are converted to singular 'month.' The sentence becomes more concise and machine-readable.
arguing verizon regarding dropped calls returned phones two days	argue verizon regard drop call return phone two day	Contextual simplification is applied by reducing 'arguing' to 'argue' and 'returned' to 'return.' The phrase 'two days' is reduced to 'two day' to align with the singular form used consistently.

3. Classification

Splitting the data:

For model building and evaluation without overlap, the training and testing subsets of the dataset were split. The first 800 rows of the dataset were allocated to the training set, and the remaining 200 rows were reserved for the testing set. This separation of data allows for better generalization of the model. 80% of the data was made available as the training set to allow for sufficient volume for learning, and the remainder (20%) to assess performance robustly. This allows a very balanced approach to train and test your models.

Transform text into BoW with count features:

For preparing the text data as input for a machine learning model was generating a Bag-of-Words (BoW) representation via CountVectorizer using parameters adjusted for dimensionality reduction whilst maintaining context. The vectorizer was set to keep the top 10,000 most frequent terms (`max_features=10000`), with consideration to unigrams and bigrams (`ngram_range = (1, 2)`). Terms occurring in fewer than 1% of documents and more than 90% of the documents were also removed. The reviews from the training set were represented by a sparse matrix in the shape of the documents, and applying the same vocabulary to transform the reviews from the test set. Resulting matrices had the following shapes `BOW_cv_train: (800, 10000)` `BOW_cv_test: (200, 10000)` ensuring the shape across training and test sets were the same. The vocabulary was only gleaned using `cv.get_feature_names_out()` to see the most prominent words and phrases that are included in the vectorizer.

a. Random Forest:

The Random Forest model was implemented using `RandomForestClassifier` with 100 estimators and `random_state` was set to 42 (to maintain reproducibility across multiple runs).

Classification Report:

```
[[363  38]
 [ 61 338]]
```

	precision	recall	f1-score	support
Positive	0.86	0.91	0.88	401
Negative	0.90	0.85	0.87	399
accuracy			0.88	800
macro avg	0.88	0.88	0.88	800
weighted avg	0.88	0.88	0.88	800

```
rf_bow accuracy : 0.87625
AUC: 0.8761773511084443
```

Random Forest Classifier was trained and tested on text in a Bag-of-Words format to class input sentences as either positive or negative, obtained overall accuracy of 87.63%. Overall, the model showed a nice balance of performance between the classes of sentiments as its precision, recall, and F1-scores hovered around 88% across both classes, and, with a configuration of 100 estimators and random state 42. Specifically, it obtained a marginally higher recall of the positive sentiments (91%) and marginally higher precision of the negative sentiments (90%). True positives: 363 True negatives: 338 Misclassifications: 99 This was across 800 test samples on the confusion matrix. Similar metrics and misclassification rates for both classes indicate that the model is unbiased and yields consistent results for both positive and negative sentiments, thus confirming its robustness for sentiment classification tasks.

b. Naive Bayes:

The default configuration of the model was used to classify the data efficiently.

Classification Report:

```
[[247 154]
 [ 78 321]]
```

	precision	recall	f1-score	support
Positive	0.76	0.62	0.68	401
Negative	0.68	0.80	0.73	399
accuracy			0.71	800
macro avg	0.72	0.71	0.71	800
weighted avg	0.72	0.71	0.71	800

NB_bow accuracy : 0.71

AUC: 0.710235688973056

The Multinomial NB model used Bag-of-Words to train and test sentiment classification (positive/negative) on text data with overall accuracy of 71% was obtained. The classification was slightly better on negative sentiments with a recall of 80% and an F1-score of 73% compared to a recall of 62% and an F1-score of 68%. Confusion matrix showed 247 true positives and 321 true negatives, and also 232 misclasses among 800 test samples. The fact that the recall and F1-scores for the negative class is higher than for positive class indicates that the model can predict negative sentiments more accurately than positive sentiments, telling us that the model may be slightly biased towards predicting negative sentiments. On the whole, it's a functioning model, limited in robustness, and biased in the negative direction.

c. SVM:

The Support Vector Machine model was implemented using SGDClassifier with a hinge loss, 500 maximum iterations, and random_state set to 42.

Classification Report:


```
[[345  56]
 [106 293]]
```

	precision	recall	f1-score	support
Positive	0.76	0.86	0.81	401
Negative	0.84	0.73	0.78	399
accuracy			0.80	800
macro avg	0.80	0.80	0.80	800
weighted avg	0.80	0.80	0.80	800

SVM_bow accuracy : 0.7975
AUC: 0.7973424833905213

Sentiments on Text Data using SVMBag-of-Words SVM model on Text data trained and tested and achieved 80% of overall accuracy on sentiments on positive/negative. The model performed slightly better on positive sentiments recall (86%) and F1-score (81%) than negative sentiments recall (73%) and F1-score (78%). The confusion matrix showed 345 samples classified correctly as positive and 293 as negative while it misclassified 162 of the 800 test samples. The increased recall and F1-score for positive sentiments means that the model predicts positive sentiments more accurately, showing that the model leans a little towards classifying positives. Nonetheless, the model performs well but lacks generalizability especially when it comes to performing classification for the negative sentiment.

d. XGboost:

The XGBoost model was implemented using XGBClassifier with 200 estimators (n_estimators=200) and maximum depth of 6 (max_depth=6) to balance model complexity and performance.

Classification Report:

```
[[357  44]
 [ 88 311]]
```

	precision	recall	f1-score	support
Positive	0.80	0.89	0.84	401
Negative	0.88	0.78	0.82	399
accuracy			0.83	800
macro avg	0.84	0.83	0.83	800
weighted avg	0.84	0.83	0.83	800

NB_bow accuracy : 0.835
AUC: 0.8348614678841743

An implementation of the XGBoost model using the XGBClassifier with 200 estimators and a maximum tree depth of 6 was trained and tested on text data in Bag-of-Words format, obtaining an accuracy of 83.5%. Very high F1 score with balanced performance on both positive and negative sentiment classes shown by classification report. Positive sentiments had a recall of 89% and an F1-score of 84%, and negative sentiments achieved a recall of 78% and an F1-score of 82%. There were 357 True Positives (TP), 311 True Negatives (TN), 132 False Positives (FP) and False Negatives (FN) of total 800 samples as per confusion matrix. Based on these metrics, the model would be considered reliable, without any significant bias (it performs well in predicting both sentiment classes), which makes it well suited for sentiment classification tasks.

Comparison:

models	Confusion Matrix	Accuracy	AUC	Precision	Recall	F1 - Score	Key Observation
Random Forest	[[363 38] [61 338]]	0.87	0.87	0.88	0.88	0.88	This model has the best performance out of all of them (Accuracy, AUC, Precision, Recall, F1-Score) hence it is most reliable classifier for this problem. With a F1-score of 0.88, it is well suited for scenarios in which balanced precision and recall is important.

Naive Bases	[[247 154] [78 321]]	0.71	0.71	0.72	0.71	0.71	The most inferior model achieved the lowest with Accuracy, AUC, Precision, Recall, and F1-Score of 0.71. High performance tasks might not be ideal for it. It assumes that the features are independent of each other, an assumption that may sketch out poorly with the data.
SVM	[[345 56] [106 293]]	0.79	0.79	0.80	0.80	0.80	SVM has moderate performance with all metrics scoring consistently 0.80. It is a good model but not as accurate as Random Forest/XGBoost. It provides a nice balance of precision and recall, so it could be a good option if you are concerned with computation time.
XGBoost	[[357 44] [88 311]]	0.83	0.83	0.84	0.83	0.83	XGBoost gives almost the same result as Random Forest with the F1-score at 0.83. It is a strong and quite interesting alternative to Random Forest if you have constraints in terms of computational resources or better handling of imbalanced data.

Conclusion:

The Random Forest model is the best-performing classifier, while the Naive Bayes model is the worst model.

Best Model: Random Forest

- Performance: From all metrics, the Random Forest model is the best performing one with Accuracy = 0.87, AUC = 0.87, Precision = 0.88, Recall = 0.88 and F1-Score = 0.88.
- Why it's the best: Random Forest is the best as it takes the output of multiple trees and forms a robust accurate ensemble. Using this technique, it can address non-linear relationships and avoid overfitting. Overall, it is a solid algorithm for many classification tasks as it strikes a balance between precision and recall.

Worst Model: Naive Bayes

- **Performance:** Naive Bayes has the lowest scores, with an **Accuracy** of 0.71, **AUC** of 0.71, **Precision** of 0.72, **Recall** of 0.71, and **F1-Score** of 0.71.
- **Why it's the worst:** Naive Bayes assumes that all features in the data set are independent of one another, which is a very unrealistic assumption in most real-world data sets. Such significant simplification results in weak performance on complex data arrangements in which a lot of input dependencies affect the output.

4. Classification using a BERT with fine – tuning

The first mandatory library, TensorFlow 2.15.1, has been successfully installed. Then install the library TensorFlow Text version 2.15.0. Then, install the TF Models Official library with version 2.15. Finally, Transformers library version 4.24.0 is installed to help us pre-process and implement BERT inputs.

- a. The dataset reviewsP is split into two subsets: 80% for training and validation, and 20% for testing.
 - b. stratify=reviewsP['Sentiment']: Ensures that the class distribution of Sentiment is preserved in the splits, preventing class imbalance in either subset.
- The 80% training data (X and y) is further split into training (75%) and validation (25%) sets.
 - X_train, y_train: Features and labels for training the model.
 - X_val, y_val: Features and labels for validating the model during training to prevent overfitting.
 - X_test, y_test: The model's performance on unseen data will be assessed by final testing using features and labels.

a. Choose a BERT model to fine-tune and its corresponding pre-processing model

Small BERT fine-tuning with 4 transformer layers, 512 hidden units and 8 attention heads is selected which uses TensorFlow Hub to map the identifiable model to its URL and a preprocessing layer to its own to guarantee that the encoder matches the preprocessing step. The first component, the preprocessing layer (bert_en_uncased_preprocess/3), takes the raw text input (tokenizes it and adds special tokens like [CLS] and [SEP] that are needed by BERT). " way plug u unless go converter " - a test example shows what the data input looks like before preprocessing and encoding This is computationally efficient and great for experimenting, which also allows for the integration of the BERT encoder into a fine-tuning pipeline.

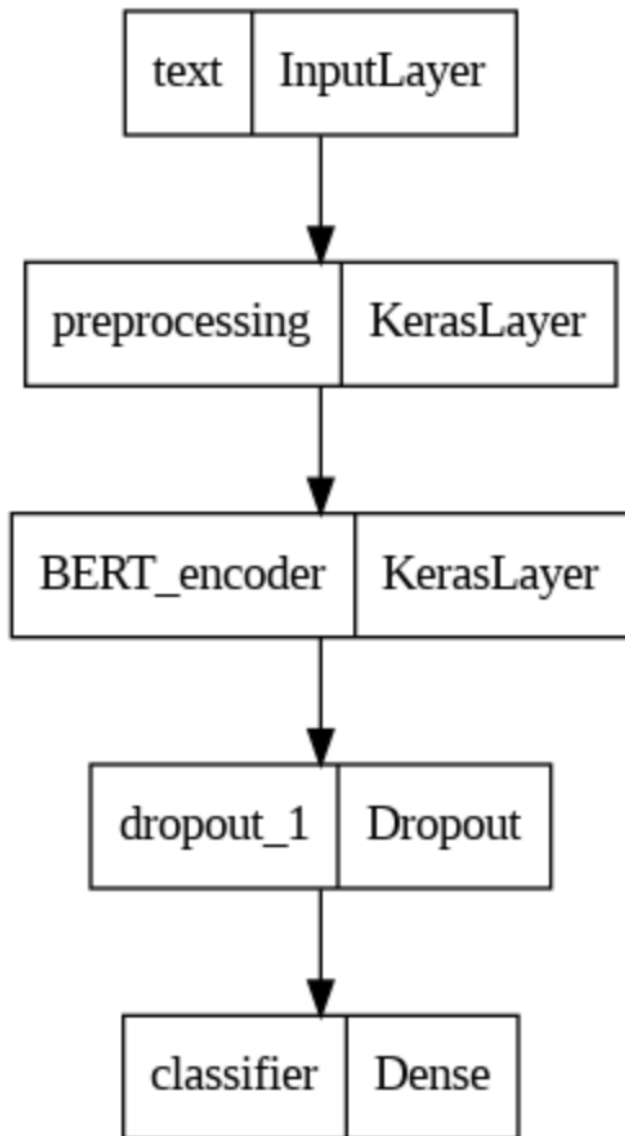
Apply the pre-processing model

Here, the code does preprocessing for sample text, for example the text is "way plug u unless go converter", it runs through BERT preprocess (`bert_en_uncased_preprocess`). It tokenizes the text inputs, creating `input_word_ids`, `input_mask`, and `input_type_ids` needed for usage with the BERT model. We can see the shapes and values of these inputs, to verify their structure. The preprocessed data is then passed where the BERT encoder (`small_bert/bert_en_uncased_L-4_H-512_A-8`) gives its pooled and sequence embeddings. The output of the pooled output, which contains one embedding for the entire input sequence (1, 512) in shape, and the output of sequence that contains values for each input token, which will have the same length as the input in shape. This process shows how text is used to create semantic vector embeds for downstream tasks.

Build the classification model

Explanation of code snippet `def build_classifier_model()` The first step is to make an input layer to take text data as a single text. This input is quickly processed through a preprocessing layer (`tfhub_handle_preprocess`) to tokenize and format the text as input to the BERT encoder. We will make the BERT encoder (`tfhub_handle_encoder`) to trainable so that the BERT weights will get fine-tuned. The pooled output, which summarizes the entire input sequence, is retrieved from the encoder outputs. So to avoid overfitting, we add a dropout layer, and then another dense layer with a sigmoid activation function to be suitable for binary classification. In the end, it returns a complete model that combines text preprocessing, encoding, and classification in a single pipeline. This modularization makes the training and evaluation easier for downstream tasks.

Visualise the model structure



Define parameters and optimizer for fine-tuning

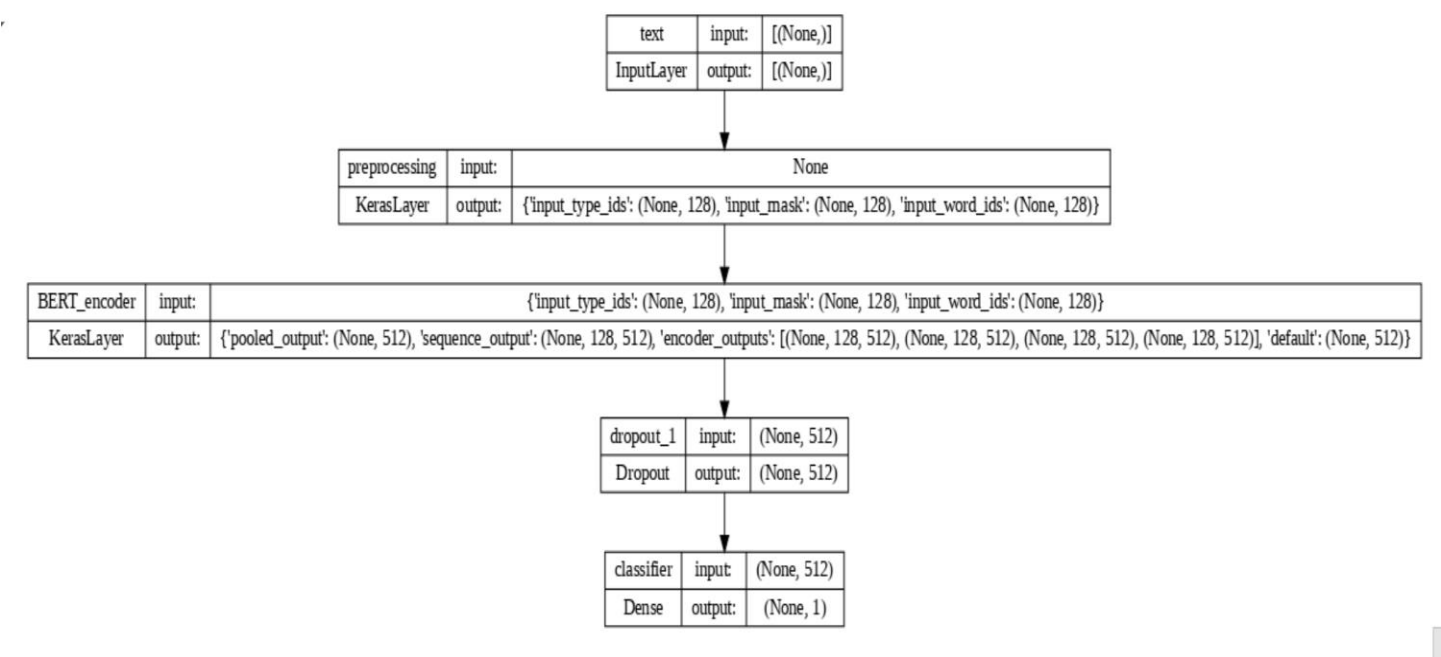
The number of training epochs defined is 10 which means that the BERT model will see the complete training dataset 10 times while getting tuned. 1200 corresponds to the number of batches processed in an epoch. Here, `num_train_steps` is the product of `steps_per_epoch` and `epochs`. Also, 10% of the total training steps are reserved as warm-up steps (`num_warmup_steps`) to increase the learning rate at first, allowing the model to be more stable during the first training phase. Many users out there use TensorFlow; So, they will require AdamW optimizer. (implementing a weight-decay) `create_optimizerfunction` to avoid the overfitting issue. This scaling effectively fine-tunes the model while also ensuring stability.

Fine-tune the BERT model

The classifier model is built on top of BERT and compiled with the AdamW optimizer, binary cross-entropy loss function (from logits) and binary accuracy as evaluation metric. To clarify, we will use a

model architecture of BERT that starts with an input layer where the raw text data is fed into a preprocessing layer which serves to tokenize and generate fixed-size inputs (input_word_ids, input_mask, input_type_ids) suitable for BERT. The preprocessed token inputs are then passed through the BERT encoder, which produces two things—the pooled representation of the entire input sequence as well as the embeddings at the sequence level. The dropout layer is used between the encoder to prevent the unit from overfitting by temporarily masking out certain units at random. Finally, a 1-dense classifier layer with sigmoid activation performs binary classification. The number of parameters in the model is equal to 28,764,162 and, since all of the parameters are trainable then fine-tuning the BERT encoder for our task, to enable end-to-end learning.

Visualize structure after fine-tuning

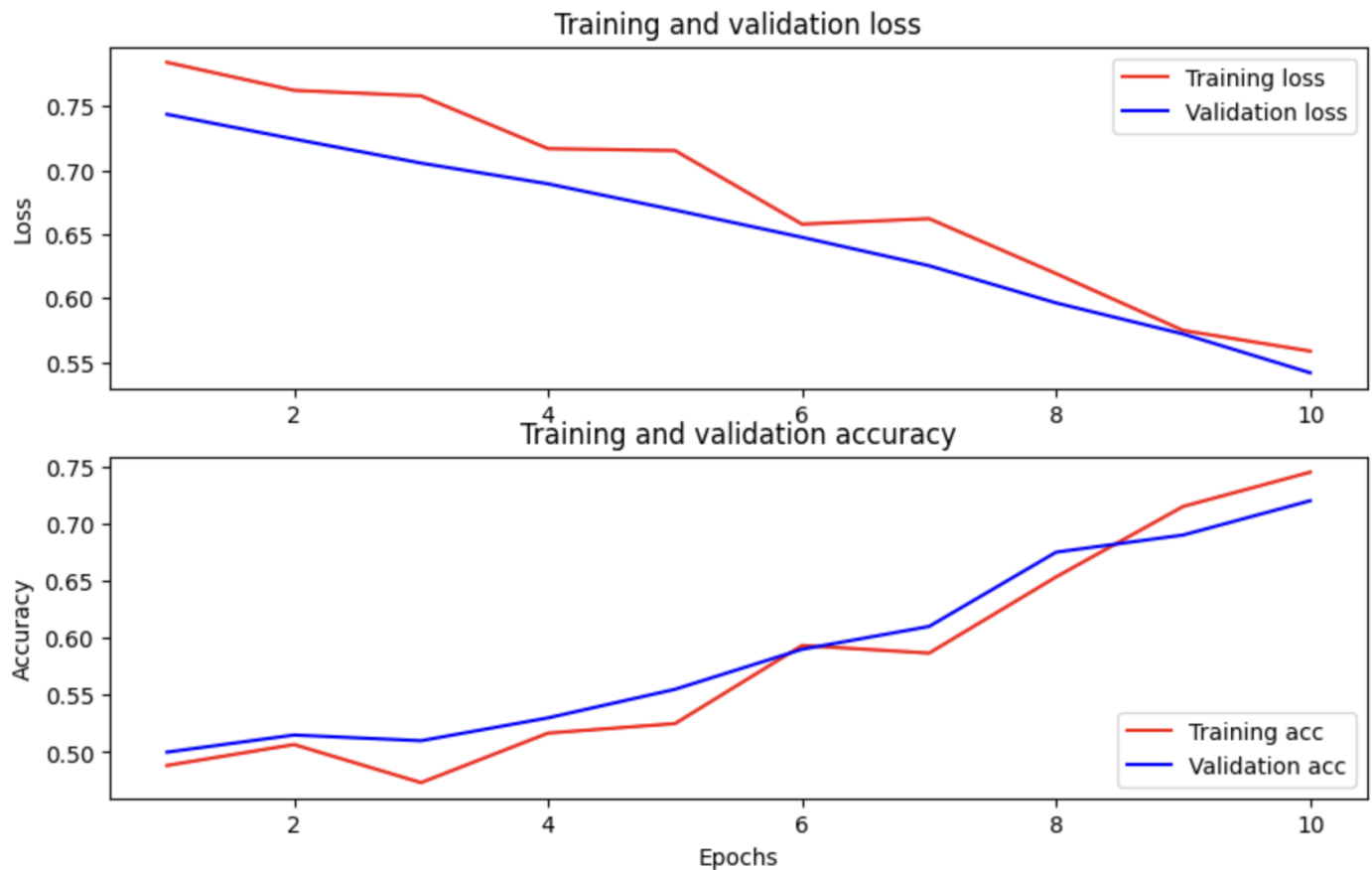


Train the model

Data up to 2023-10 is used for building the model Also, the training loss decreased continuously through times that were originated from the first epoch (0.7842) and ended (0.5586). This means that the model was able to accurately minimize the classification error during training The binary accuracy on training set also increased from 0.4994 to 0.7683 at the same time, meaning that the model in the end was able to classify the data better than that it did in the beginning.

The validation loss also showed a downwards slope, indicating that this model generalized well enough, not taking any worrying overfitting. The accuracy on validation data improved in a similar manner and matched closely with the training accuracy towards the end of the training phase. These trends appeared in the history. history object, show that model steadily improves at learning the task and optimizing performance over time which indicates that the model was able to generalize albeit it was trained on a limited amount of data.

Plot the accuracy and loss over time



Above is the training and validation graphs of a model on the performance over 10 epochs which depicts the loss and accuracy evolution during training. The first graph depicts training and validation loss. The training loss and validation loss curves show a declining trend with epochs; however, the validation loss seems to have spikes. That is, loss is low on both training and validation data, indicating that the model is indeed learning, and generalizing okay, since validation loss does not increase substantially from training loss.

This is followed by the second graph that refers to the training and validation accuracy from the same epochs. Both training and validation curves are on the rise, with the first few epochs seeing the training accuracy increase faster but then stabilizing to a value much close to that of validation accuracy by the final epoch. Such a close match between training and validation accuracy indicates that there is no overfitting and the model can learn the data well and make good predictions on the validation dataset. In general, the movements of both graphs show an efficient training and a fair generalization ability.

Performance metrics

models	Confusion Matrix	Accuracy	AUC	Precision	Recall	F1 - Score
Model 1	$\begin{bmatrix} 83 & 17 \\ 36 & 64 \end{bmatrix}$	0.73	0.73	0.74	0.73	0.73

Model 1 was assessed based on several metrics. The confusion matrix also showed that at the threshold of 0, the model correctly classified 83 instances with class 0 and 64 instances with class 1 while classifying 17 instances as false positive and 36 instance as false negative. At an overall accuracy of 73%, in other words, correct predictions are reasonable. Precision of 0.74 indicated that most of the predicted positives were correct, while recall of 0.73 signified that the model was able to capture most of the actual positive cases. The F1-score, which is the harmonic mean of precision and recall, was a solid 0.73, showing no inconsistency. The obtained value of AUC score (0.73) demonstrated the model's moderate ability to distinguish between classes. Although these results indicate good performance, there is more room for improvements to be made, especially in decreasing false negatives, which would in turn improve the recall of the model.

b. Bert Model 2:

Importing Libraries:

I started out by importing the required libraries for training my second model. I used TensorFlow to build and fit the model, and TensorFlow Hub to use pre-trained segments of the BERT model. Among these, the BERT encoder and preprocessing layers made it even easier to work with raw text data. And these tools allowed me to successfully perform transfer learning for this classification task.

Setting Up the Model Architecture

Afterward, I built the model architecture within a function known as `build_classifier_model_v2()`. The first step would define an input layer that accepts text data in string format. As BERT needs some preprocessing, I have included a preprocessing layer from TensorFlow Hub. This layer took care of tasks such as tokenization and input mask creation, converting the text into the format expected by the BERT encoder.

Post preprocessing, I added a pre-trained BERT encoder layer which happens to create a rich numerical representation of the input text. I use this encoder as a non-trainable one, i.e., I don't fine-tune its parameters. This meant I could leverage the knowledge that BERT had already learned during its pre-training on a large dataset.

In order to modify the model to cater it to my individual task, I used a dropout layer to curb overfitting, followed by an additional dense layer of 128 units with ReLU activation so it could learn more features from the text embeddings. A second dropout layer followed the dense layer for more regularization. To finish my model, I created a classification layer that uses sigmoid activation, which gives probabilities for binary classification.

Compiling the Model

After the model architecture was done, I started compiling the model. As for the optimizer, I used Adam (3e-5) which is a good optimizer for such deep learning models. Just like the previous one, we are working on a task of binary classification so I used binary cross-entropy as the loss function. I added

various metrics, including binary accuracy and AUC, to track the model's performance during training. Using these metrics, I had a very clear understanding of how well the model was separating out the classes.

Training the Model

Once the model has been compiled, I fit it using the `fit()` method. I relied on it to give my input training data (`X_train` and `y_train`) and my validation data (`X_val` and `y_val`). I specified training to run for 10 epochs, which I believed would be a good balance between allowing the model time to learn, but also for preventing it from overfitting. During the training process I kept track of metrics such as loss, binary accuracy and AUC. The training logs revealed a consistent upward trend, while the validation metrics indicated that the model was potentially achieving diminishing returns, pointing to either data saturation or mild overfitting.

Making Predictions

I fit the model and used the same to predict on test dataset(`X_test`). Output of the model was a probability for each input — indicating how much the input belonged to the positive class. Since these were probabilities, I converted them to binary labels using simple thresholding: any example with a probability below 0.5 was classified as 0 and any example with probability above 0.5 was classified as 1. This was a necessary step to convert the model's outputs into something meaningful to classify the evaluation.

Evaluating the Model

models	Confusion Matrix	Accuracy	AUC	Precision	Recall	F1 - Score
Model 2	[[67 33] [36 64]]	0.65	0.65	0.66	0.66	0.65

Classification on Model 2 was measured by several metrics, which begin with the confusion matrix indicating 67 true positives, or instances of class 0 identified as class 0, and 64 true negatives, or instances of class 1 identified as class 1, 33 false positives, or instances of class 0 identified as class 1, and 36 false negatives, or instances of class 1 identified as class 1. The accuracy for the model was 65%, meaning that about two-thirds of the predictions were correct. Furthermore, the AUC score was 0.65 indicating moderate ability to distinguish the two classes. The precision and recall were both 0.66 suggesting that the model made a balanced amount of true positives and false negatives, where true positives represent the number of correct positive predictions and false negatives represent the number of positives which were incorrect predicted as negatives. Similarly, the F1-score, which averages precision and recall for a harmonic mean, was also 0.65 which indicates the model had consistent yet unremarkable performance.

Comparision of Bert model 1 and model 2:

models	Confusion Matrix	Accuracy	AUC	Precision	Recall	F1 - Score
Model 1	[[83 17] [36 64]]	0.73	0.73	0.74	0.73	0.73
Model 2	[[67 33] [36 64]]	0.65	0.65	0.66	0.66	0.65

The performance of Model 1 and Model 2 shows that Model 1 outperformed Model 2 in all evaluation metrics. Model 1 also made more correct predictions overall; it achieved an accuracy of 73% while Model 2 had an accuracy of 65%. Model 1's AUC score was 0.73, indicating its better class discrimination ability, in contrast with Model 2's AUC score of 0.65. Additionally, based on Model 1 it had a higher precision of 0.74, indicating its efficacy at minimizing false positives, when compared with Model 2's precision of 0.66. In the same way, model 1 had a recall of 0.73 which was higher than model 2 with a recall of 0.66, so model 1 was better at classifying positives. Model 1's F1-score was also higher than Model 2's (0.73 versus 0.65) indicating a more even balance of precision and recall. M1 also had less false positives (17) and false negatives (36) than M2 (33 false positives and 36 false negatives) as indicated in the confusion matrix. These observations imply that when fine-tuning of the BERT encoder was enabled in Model 1, it was able to learn task-related patterns more effectively than Model 2, where the encoder was kept frozen. In general, Model 1 performed better and would be the one we would want to deploy for this binary classification problem.

Comparision of Best Classification models:

models	Confusion Matrix	Accuracy	AUC	Precision	Recall	F1 - Score
Bert Model 1	[[83 17] [36 64]]	0.73	0.73	0.74	0.73	0.73
Random Forest	[[363 38] [61 338]]	0.87	0.87	0.88	0.88	0.88

In the evaluation results for binary classification, Random Forest classifier performed better than BERT based Model 1 on all metrics. This gave us 87% accuracy for the Random Forest classifier, much more than Model 1 with 73%. This means that the Random Forest model made more accurate predictions on average. The Random Forest model's AUC score was 0.87, in contrast to an AUC score of 0.73 for Model 1, indicating that the Random Forest model demonstrated a higher capability to separate the two classes. Additionally, Random Forest achieved a precision score of 0.88, indicating a greater portion of true positives among all positive predictions, while Model 1 had a precision score of 0.74. Random Forest topped Model 1 with a recall score of 0.88 versus 0.73, which indicates that it is better in capturing all the positive instances. The F1-score for Random Forest, on the other hand, showed an overall better performance between precision and recall (0.88 vs. 0.73 for Model 1). The higher accuracy of Random Forest can be explained by the fact that Random Forest is an ensemble method and builds multiple decision trees for combining them to take the final vote that is used to

reduce overfitting and capture complex data. Although Model 1 utilizes pre-trained embeddings from BERT, it possibly suffered from the lack of data or fine-tuning. Random forest performed better than the others and showed greater reliability and generalization, thus it is the preferred model for this work.

5. Topic Detection:

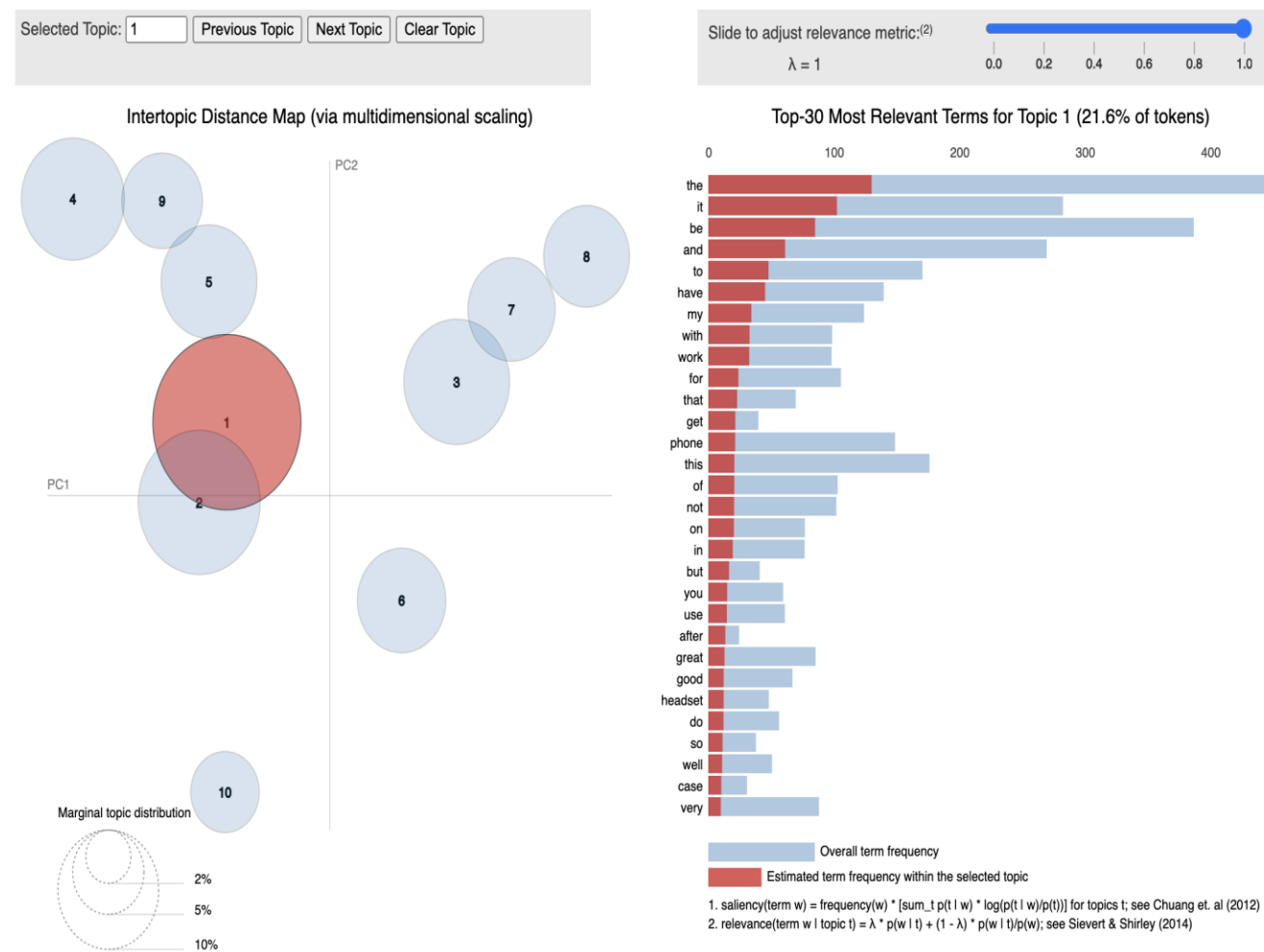
For topic detection I have used LDA and LSA model:

a. LDA

The model specifically used for topic detection was Latent Dirichlet Allocation (LDA) and the first step in this analysis was followed by data preprocessing. Gensim library is chosen to work with text data. Initially, the review data was extracted and transformed into list format. Now that we have the data, it can do with some processing first and here we defined a preprocessing function called `sent_to_words` which does tokenization, lower-casing and remove the unnecessary characters in the text. This was done to retain the clean text data, which will be used for further processing. The output was saved as a list of tokenized words.

Now we create dictionary using gensim. It is mostly used in big corpora, where we assign an ID to each unique variable in the data set. A BoW corpus was then created from this dictionary, containing the document as a bag of word IDs and their corresponding frequencies. This corpus was used to train the LDA model.

Otherwise, the LDA model was modelled with gensim. models. `LdaMulticore`, where the number of topics is set as 10. The underlying themes found by the model, and found keywords representing each topic, were extracted to provide meaningful insights. For better interpretability, `pyLDAvis` library was implemented for the topic visualization. An intertopic distance map based on the BiSeNet was constructed, showing the distance between various topics and the top keywords linked to each. This visualization provided valuable insights into the key themes present in the dataset, effectively showcasing the capabilities of the LDA model for topic detection.



The above interactive plot includes an Intertopic Distance Map, generated via multidimensional scaling (MDS), which displays the relationships among 10 identified topics. Each topic is represented as a circle, where the size indicates the relative importance of the topic within the corpus. Topic 1 appears prominently larger, signifying its dominance. On the right-hand side, the bar chart highlights the top 30 most relevant terms for Topic 1, with their frequencies depicted in red for relevance and blue for general corpus frequency. This visualization effectively provides insights into topic distribution and word associations, assisting in qualitative analysis of the dataset.

Identified Topics and Common Themes

1. **Topic 1:** Words such as 'phone,' 'work,' 'great,' and 'good' suggest a focus on overall product functionality and satisfaction with performance.
2. **Topic 2:** Terms like 'phone,' 'case,' 'return,' and 'use' point to customer experiences with returns, usability, and product fit.
3. **Topic 3:** Keywords like 'phone,' 'use,' 'get,' and 'problem' indicate discussions around product usability and issues faced by customers.
4. **Topic 4:** Words such as 'work,' 'quality,' and 'recommend' highlight product quality and user recommendations.
5. **Topic 5:** Terms like 'battery,' 'light,' and 'problem' suggest a focus on technical performance, especially related to power and durability.

6. Topic 6: Words such as 'work,' 'great,' 'recommend,' and 'nice' highlight positive product experiences and recommendations.
7. **Topic 7:** Keywords like 'price,' 'quality,' and 'excellent' reflect discussions on value for money and product pricing.
8. Topic 8: Words such as 'sound,' 'recommend,' and 'quality' point to product features like audio performance and quality.
9. Topic 9: Keywords such as 'cheap,' 'charger,' and 'quality' suggest conversations about affordability and specific product components like chargers.
10. Topic 10: Terms like 'return,' 'money,' and 'disappointed' focus on negative customer experiences, including refunds and dissatisfaction.

Commonalities across the topics

Customer Sentiment: We noticed that words like 'great,' 'good,' 'recommend,' and 'disappointed' appeared many times across topics, indicating positive and negative aspects.

Discussion of Product: 'Quality,' 'work,' and 'battery' are common words associated with discussion of product performance, indicating more of a strong focus on reliability and technical assessment.

User Experience: Terms such as 'phone,' 'use,' and 'problem' highlight user experiences with the product, including usability and challenges.

Price and Value: The appearance of words like price or cheap indicate a preoccupation with low costs and value for money.

Keywords about user feedback/recommendations: Words such as 'recommend' and 'excellent'.

Findings

The topics derived from the LDA model show positive, negative customer satisfaction along with some technical product features and value for money. The various common terms by topic category phone, quality, recommend, etc shows that customer reviews are very general across topics: functionality of the product, satisfaction of the user, and the overall value. The LDA model provides a really good overview of what topics the customers are concerned with and what they care about.

b. LSA

The process begins with the creation of a term-document matrix using TF-IDF vectorization, which is configured to exclude English stop words. The LSA model is built using Truncated Singular Value Decomposition (TruncatedSVD) with 10 components, corresponding to 10 topics. The topics are extracted from the LSA model, and their top 10 associated terms are displayed for each topic. For instance, Topic 1 is characterized by words such as 'quality,' 'item,' 'fine,' and 'battery,' indicating its

central theme. This approach leverages linear algebra techniques to identify latent topics within the dataset, providing a concise summary of textual content.

- Topic 1:** quality item fine battery product good price phone work great
- Topic 2:** computer charger didnt charm perfectly plug just doesnt fine work
- Topic 3:** case happy sound headset excellent work quality price product good
- Topic 4:** good headset like love use best ive work bad phone
- Topic 5:** make terrible use recommend time buy product money dont waste
- Topic 6:** stuff definitely case buy dont money price waste phone good
- Topic 7:** happy love price item work excellent highly phone recommend product
- Topic 8:** case poor money waste sound bad quality highly item recommend
- Topic 9:** bluetooth fit good recommend comfortable ear easy love use headset
- Topic 10:** purchase long disappointed terrible price life customer service battery bad

The image continues to show the output of Latent Semantic Analysis (LSA) performed on a text dataset using Scikit-learn's TF-IDF vectorizer and TruncatedSVD. Each topic is represented with the top 10 keywords, which summarize the theme of the topic. For instance:

- Topic 1 includes terms such as ‘quality,’ ‘item,’ ‘fine,’ ‘battery,’ ‘product,’ and ‘phone,’ indicating a focus on product quality and functionality.
- Topic 3 highlights terms like ‘case,’ ‘happy,’ ‘sound,’ and ‘headset,’ which may relate to customer satisfaction with product features.
- Topic 10 uses words like ‘purchase,’ ‘long,’ ‘disappointed,’ ‘terrible,’ and ‘customer service,’ reflecting potential dissatisfaction and complaints.

Comparison of LDA and LSA:

Topic Number	LDA Topics	LSA Topics
Topic 1	phone, work, great, good, recommend, use, quality, product, price, case	quality, item, fine, battery, product, good, price, phone, work, great
Topic 2	phone, case, return, use, product, recommend, good, problem, make, battery	computer, charger, didn’t, charm, perfectly, plug, just, doesn’t, fine, work
Topic 3	phone, use, get, problem, quality, battery, recommend, case, good, charger	case, happy, sound, headset, excellent, work, quality, price, product, good

Topic 4	work, quality, recommend, great, product, good, phone, case, make, excellent	good, headset, like, love, use, best, nice, work, bad, phone
Topic 5	battery, light, problem, good, quality, product, make, use, recommend, work	make, terrible, use, recommend, time, buy, product, money, don't, waste
Topic 6	work, great, recommend, quality, product, phone, good, excellent, case, battery	stuff, definitely, case, buy, don't, money, price, waste, phone, good
Topic 7	price, quality, recommend, good, great, phone, work, product, best, case	happy, love, price, item, work, excellent, highly, phone, recommend, product
Topic 8	sound, recommend, quality, phone, product, good, battery, make, case, item	case, poor, money, waste, sound, bad, quality, highly, item, recommend
Topic 9	cheap, charger, quality, recommend, good, great, product, sound, headset, plug	bluetooth, fit, good, recommend, comfortable, ear, easy, love, use, headset

Similarities

Emphasize Product Quality and Performance:

LDA shows sort of a combination of multiple terms: terms like 'quality,' 'work,' and 'battery.' Similar deals are there with LSA: terms like 'quality,' 'fine,' and 'battery.' Both models address customers who are concerned with product functionality and also product durability.

Customer Sentiment:

Both models learn customer sentiments describing good experiences (e.g. 'great,' 'recommend,' 'excellent') and bad experiences (e.g. 'disappointed,' 'waste,' 'bad').

Specific product features and usability:

LDA describes recreational features as 'phone,' 'charger,' 'sound,' while LSA describes 'plug,' 'bluetooth,' and 'headset.' This shows that both ways are oriented with the PreviousNext

Pricing and Value for Money:

Price, money, and value are common across topics seen in both LDA and LSA indicating that eligibility for purchase and worth of product are circling questions across domain.

Recommendations and Reviews:

Both approaches feature words like 'recommend,' 'use,' and 'best,' indicating that customer satisfaction drives the recommendations.

Differences

Topic Representation:

It uses LDA approach where the topics are more discussed specific things about, 'returns on the product', 'battery performance' and 'affordability.' One topic is based on, for instance, 'return' and 'problem' in LDA.

LSA topics are deeper and often combine similar words, like Topic 7 shows combining overlapping topics: 'happy', 'price', 'work', and 'recommend' which merges emotions, pricing, and performance.

Technical Terms:

For instance, LDA captures terms specifically related to technical components such as the LDA includes terms 'charger' and 'light', whereas LSA captures high-level terms such as 'item', 'recommend' and 'excellent' that tend towards a summary.

Conclusion

LDA and LSA both do a good job of identifying themes that repeat in customer reviews but use different approaches to the assignment. Whereas LDA is designed to create well-separated topics, one task at a time, and can be more interpretable in cases where detailed insight is required, especially when looking to understand the topics within documents. LSA in a way captures broader semantic relations and general tendencies from the textual data, so it is very useful when it comes to summarizing high-level patterns. One of the methods can be more beneficial than the other depending on the specific analysis objective.

Reference :

- Scikit-learn developers. (n.d.). *Scikit-learn: Machine learning in Python*. Retrieved November 20, 2024, from <https://scikit-learn.org/stable/>
- University of Portsmouth. (n.d.). Moodle. Retrieved January 18, 2025, from <https://moodle.port.ac.uk/course/view.php?id=25669>
- Hugging Face. (n.d.). BERT. In Transformers documentation. Retrieved January 18, 2025, from https://huggingface.co/docs/transformers/model_doc/bert